UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

INGENIERÍA INFORMÁTICA

Sistema vía correo electrónico para la gestión de convocatorias de auxiliaturas

Proyecto 1 de Tecnología Web

Grupo 09 SA

Registro	Integrante	Carrera
223045888	Acosta Osorio Lin Fernanda	Ingeniería en Sistemas
223043982	Ramos Vargas Brayan Joel	Ingeniería en Sistemas
213114976	Rodriguez Aguilera Walter	Ingeniería en Sistemas

Santa Cruz de la Sierra, octubre de 2026

Resumen ejecutivo

Desarrollamos un servicio Java que recibe órdenes en el asunto de correos enviados a la cuenta del Grupo 09 SA. Validamos sintaxis, parámetros y permisos. Las operaciones privadas o de modificación requieren un código enviado al correo del solicitante antes de ejecutar la orden en PostgreSQL. Las consultas públicas responden directamente.

La comprobación integral se realizó el 12 de octubre de 2026. Un correo externo solicitó el catálogo de ayuda y envió LISCON["*"]; el servicio leyó ambos mensajes, procesó la consulta en la base de datos y respondió al mismo Gmail con el asunto ÉXITO - Orden procesada. Gmail colocó las respuestas en Spam por tratarse de un remitente institucional nuevo; la entrega y el contenido se verificaron.

1 Objetivo y alcance

El objetivo consiste en automatizar la gestión de convocatorias de auxiliaturas mediante comandos enviados por correo electrónico. El programa se ejecuta continuamente en Java, revisa la bandeja cada diez segundos y no requiere una interfaz gráfica.

- Interpretar órdenes con el formato COMANDO["parámetro", ...].
- Controlar permisos para Decano, Vicedecano, Evaluador y Postulante.
- Persistir usuarios, convocatorias, cargos, postulaciones, expedientes, evaluaciones, resultados y auditoría.
- Responder al remitente con un mensaje de éxito o una explicación del error.
- Evitar el reprocesamiento mediante identificadores persistentes obtenidos por Roundcube HTTPS.

2 Arquitectura de la solución

Componente	Tecnología	Responsabilidad
Cliente externo	Gmail u otro correo	Envía la orden en el asunto y recibe la respuesta.
Entrada	Roundcube HTTPS	Lista la bandeja con identificadores estables y tráfico cifrado.
Servicio	Java 17 y Maven	Analiza, valida, autoriza y coordina cada orden.
Persistencia	PostgreSQL	Ejecuta operaciones transaccionales y conserva auditoría.
Salida	Roundcube institucional	Entrega la respuesta desde el servidor Tecnoweb.
Empaquetado	Maven Shade	Genera un JAR ejecutable con sus dependencias.

Flujo de ejecución: recepción y registro durable, análisis del asunto, validación, solicitud de confirmación cuando corresponde, verificación del código y del rol vigente, operación SQL transaccional y envío de la respuesta. Ante un corte, conservamos la cola y reintentamos sin repetir el cambio de negocio.

3 Casos de uso implementados

Caso	Cobertura implementada	Comandos
CU1	Alta y listado de usuarios; roles y activación.	INSUSU, LISUSU
CU2	Convocatorias y cargos de auxiliatura.	INSCON, LISCON, INSCAR, LISCAR
CU3	Postulación múltiple y expediente digital.	INSPOST, LISPOST
CU4	Evaluación de requisitos y estado de habilitación.	EVAREQ, HABPOS
CU5	Méritos, examen y asignación de evaluadores.	REGMER, REGEXA, ASIEVA
CU6	Ranking por puntaje y asignación de plazas.	GENRAN, ASIPLA
CU7	Registro y consulta de acciones realizadas.	AUDCAL
CU8	Cantidad y promedio por estado.	REPEST

4 Catálogo de órdenes

Orden	Parámetros	Rol requerido
INSUSU	registro, nombres, apellidos, rol, teléfono, correo, activo	Decano
LISUSU	filtro	Decano
INSCON	código, título, inicio, fin, estado, plazas	Decano
LISCON	filtro	Público
INSCAR	nombre, unidad, activo	Decano
LISCAR	filtro	Público

Orden	Parámetros	Rol requerido
INSPOST	convocatoria ID, cargo ID, ruta del expediente	Postulante
LISPOST	filtro	Postulante
EVAREQ	postulación ID, requisito, cumple, observación	Evaluador
HABPOS	postulación ID, estado, observación	Decano
REGMER	postulación ID, criterio, puntaje, observación	Evaluador
REGEXA	postulación ID, puntaje, observación	Evaluador
ASIEVA	postulación ID, evaluador ID, tipo	Decano
GENRAN	convocatoria ID	Decano
ASIPLA	postulación ID, asignar true o false	Decano
AUDCAL	entidad, entidad ID	Decano
REPEST	convocatoria ID	Decano
AYUDA	sin parámetros	Público
CONFIRMAR	código de un solo uso recibido por correo	Solicitante original

5 Reglas de validación

- Escribimos los comandos en mayúsculas como convención. El analizador también reconoce minúsculas y espacios alrededor del comando. Cada parámetro va entre comillas dobles y no admite espacios accidentales en sus extremos.
- El analizador rechaza asuntos incompletos, comas finales, parámetros sin comillas y comandos desconocidos.
- Las fechas usan el formato ISO AAAA-MM-DD y la fecha final no puede preceder a la inicial.
- Los identificadores deben ser enteros positivos; las plazas aceptan cero o un número positivo. Letras, NaN, infinitos y desbordamientos se rechazan.
- Los puntajes se limitan al intervalo de 0 a 100 con máximo dos decimales. PostgreSQL repite esta comprobación mediante restricciones CHECK.
- Los textos respetan el tamaño de cada columna y no admiten controles, saltos de línea ni espacios accidentales en los extremos.
- Las operaciones privadas verifican el correo activo y el rol; el Vicedecano puede ejecutar operaciones administrativas del Decano.

- El primer DECANO requiere la tabla de usuarios vacía, coincidencia entre remitente, correo nuevo y BOOTSTRAP_ADMIN_EMAIL, y confirmación del código recibido en esa cuenta.

6 Modelo de datos

Tabla	Contenido principal
p1_usuario	Registro, nombres, rol, correo y estado.
p1_convocatoria	Periodo, estado y cantidad de plazas.
p1_cargo	Cargo y unidad académica.
p1_postulacion	Relación entre convocatoria, postulante y cargo; puntajes y plaza.
p1_expediente	Ruta y tipo del documento digital.
p1_evaluacion_requisito	Cumplimiento, observación y evaluador.
p1_merito / p1_examen	Calificaciones que alimentan el resultado final.
p1_asignacion_evaluador	Evaluadores asignados a cada postulación.
p1_auditoria	Actor, comando, entidad, detalle y fecha.
p1_correo_control	Estado, reintentos y respuesta pendiente.
p1_resultado_orden	Resultado idempotente transaccional.
p1_correo_procesado	UID de los correos respondidos.
p1_confirmacion	Hash del código, orden original, solicitante, vencimiento y consumo.

Las tablas usan el prefijo p1_ para convivir con otros proyectos del grupo sin alterar sus datos. Las consultas emplean sentencias preparadas. Cada operación, su auditoría y su resultado idempotente se confirman en una sola transacción; una caída posterior del correo no vuelve a ejecutar el cambio.

7 Cálculo y ranking

Cada registro de méritos o examen actualiza los acumulados de la postulación. El resultado final se calcula con 40 % de méritos y 60 % del promedio de examen. GENRAN ordena de mayor a menor puntaje y utiliza el identificador de postulación como desempate estable.

ASIPLA con true exige habilitación y disponibilidad de plazas y cambia el estado a GANADOR. Con false retira la plaza y devuelve un GANADOR a HABILITADO sin habilitar una postulación rechazada. REGEXA exige una postulación habilitada o ganadora y un evaluador asignado.

8 Seguridad y manejo de errores

- Las contraseñas se reciben mediante MAIL_PASSWORD y DB_PASSWORD; no se incluyen en el repositorio ni en los paquetes.
- El servicio valida roles, sintaxis, tipo, rango, fechas, estados y tamaño antes de ejecutar SQL; PostgreSQL conserva restricciones defensivas equivalentes.
- Confirmamos cada operación sensible con CONFIRMAR["codigo"]. El código aleatorio autoriza solo la orden guardada, se vincula al solicitante y se consume en la misma transacción que el negocio. Volvemos a verificar su rol al confirmar.

El código tiene un plazo de recepción de 30 minutos. Una confirmación registrada a tiempo conserva su validez mientras espera en la cola. Rechazamos códigos incorrectos, vencidos, reutilizados y remitentes que no coinciden. La cola difiere las ráfagas y no descarta definitivamente una orden por saturación temporal.

- Cada ciclo procesa como máximo 25 mensajes. Los IDs históricos se consultan por lotes y no se cargan todos en memoria.
- Roundcube HTTPS cifra autenticación, lectura y respuesta; POP3 sin TLS queda solo como respaldo manual.
- Los cortes de PostgreSQL, Internet o Roundcube activan tiempos máximos y reintentos progresivos, sin cerrar el servicio.
- La cola p1_correo_control separa ejecución y respuesta: si el correo está caído, la respuesta queda pendiente sin repetir el negocio.
- p1_resultado_orden usa el identificador del mensaje como clave idempotente. Negocio, auditoría y resultado se confirman o revierten juntos.
- Un bloqueo asesor de PostgreSQL impide que dos instancias procesen la misma bandeja simultáneamente.
- Windows y Linux incluyen scripts de reinicio; Fedora dispone además de una unidad systemd endurecida.
- El programa conserva las tablas ajenas al Proyecto 1 y trabaja únicamente sobre el esquema p1_.

9 Pruebas realizadas

Prueba	Resultado observado
Compilación Maven	JAR ejecutable generado correctamente.
Pruebas automatizadas	44 pruebas con cero fallos, errores y omisiones; incluye PostgreSQL 15 local y el flujo de los ocho casos de uso.
Fuzzing	10.000.000 de asuntos aleatorios sin derribar el analizador.
Concurrencia	320.000 análisis en 32 hilos sin errores.
PostgreSQL remoto	Esquema resiliente p1_ y restricciones CHECK aplicados correctamente.

Prueba	Resultado observado
Caída simulada	El proceso permaneció activo y aplicó reintentos progresivos.
Roundcube HTTPS	Lectura real verificada sobre 149 mensajes sin exponer contenido.
Filtro real	LISCON["C"] ejecutó SQL parametrizado y respondió: Sin resultados.
Dependencias	Jackson 2.22.3, JDBC 42.7.14, HttpClient 5.6.1 y Angus Mail 2.0.4.

La suite actual comprende 44 pruebas automatizadas en Windows con Java 17 y PostgreSQL 15 aislado, sin fallos ni pruebas omitidas. Incluye el recorrido de los ocho casos de uso con confirmaciones y datos temporales. Las pruebas institucionales por correo registradas previamente corresponden a AYUDA y LISCON. La migración y el arranque de esta revisión en el servidor institucional todavía requieren comprobación con sus credenciales.

10 Instalación y ejecución

Requisitos de ejecución: Java 17, conexión a los servicios institucionales y credenciales del grupo. Incluimos el JAR con sus dependencias en dist. Maven 3.9 o superior solo es necesario para recompilar o ejecutar pruebas.

Paso	Acción
1	Aplicar database/schema.sql en db_grupo09sa.
2	Definir MAIL_PASSWORD, DB_PASSWORD y BOOTSTRAP_ADMIN_EMAIL sin publicar claves.
3	Usar dist/inf513-proyecto1-1.0.0.jar. Para reconstruir: mvn clean verify.
4	Windows: scripts/run-service.ps1. Linux: scripts/run-service.sh o systemd.
5	Solicitar el catálogo de ayuda y revisar también Spam.

11 Demostración para la defensa

Minutos	Demostración
0–2	Problema, objetivo y formato del asunto.
2–5	Arquitectura y modelo de datos.
5–8	Arranque del servicio Java y catálogo de ayuda.
8–12	Crear o listar datos de convocatoria y postulación.
12–15	Calificaciones, ranking y estadísticas.
15–18	Auditoría, manejo de errores y conclusión.

12 Conclusiones

Cumplimos el flujo solicitado por el docente: una orden enviada en el asunto se procesa automáticamente, afecta o consulta PostgreSQL y genera una respuesta al remitente. La implementación mantiene separados los datos, valida la entrada y conserva una cola transaccional para recuperarse de cortes sin repetir operaciones.

La prueba institucional anterior verificó la entrega de respuestas a Gmail para las consultas públicas. Esta revisión añade confirmación por correo y controles de cola, conexión y habilitación. Entregamos código, esquema, informe, presentación y manual concordantes. Antes de la defensa debemos aplicar la migración y comprobar el servicio actualizado, revisar Spam y usar los identificadores reales de los registros.